

De Firebase a Microsoft SQL Server

Destinatario: Departamento de TI / Encargado de Informática

Proyecto: SmartAssign (Gestión de Personal y Planificación de Planta)

Fecha: Junio 2026

1. Arquitectura y Contexto Actual

Actualmente, la aplicación de gestión de personal y planificación de planta *SmartAssign* funciona utilizando **Firebase** (Google Cloud Platform) como infraestructura de datos y autenticación de usuarios. Este diseño inicial ha aportado importantes ventajas:

- **Base de Datos NoSQL (Firestore):** Permite un almacenamiento flexible de colecciones como operarios, líneas, turnos y mermas sin esquemas rígidos.
- **Sincronización en Tiempo Real:** Firebase actualiza la pantalla del coordinador e inspectores instantáneamente ante cualquier cambio de estado (ej: reasignaciones o incidencias de planta).
- **Autenticación Integrada (Firebase Auth):** Gestión segura de usuarios y sesiones directamente integrada en la app.
- **Persistencia Local (Offline Sync):** Sincronización automática de datos si las tabletas o móviles pierden conectividad en zonas grises de la planta.

Motivación de la Migración: Para integrar la aplicación dentro de la infraestructura local/on-premise de la empresa y dar cumplimiento a las políticas corporativas de almacenamiento (donde toda la información debe centralizarse y auditarse en **Microsoft SQL Server**), se propone migrar el backend conservando las funcionalidades críticas de la app actual.

2. Nueva Arquitectura Propuesta (Capa Intermedia)

Conectar una aplicación móvil o web híbrida directamente a Microsoft SQL Server (puerto 1433) representa un grave riesgo de seguridad y rendimiento. Por tanto, la migración requiere implementar una **Arquitectura de 3 Capas**:

1. **Capa de Presentación (Frontend):** La aplicación React actual compilada para Web e instalada como app nativa Android vía Capacitor en los dispositivos de planta.
2. **Capa de Negocio (Backend API):** Un servidor intermedio dedicado (desarrollado en **Node.js (Express/Fastify)** o **.NET 8 Core**) que expone endpoints REST a la app y se conecta internamente a la base de datos.

3. **Capa de Datos (Base de Datos):** Servidor corporativo de **Microsoft SQL Server** hospedado en la red privada de la empresa.

Simulación de Tiempo Real: Para mantener las notificaciones y actualizaciones en vivo que ofrecía Firebase, la API intermedia incorporará soporte para **WebSockets** (usando *Socket.io* en Node.js o *SignalR* en .NET). Esto notificará de inmediato a todos los clientes conectados cuando ocurra un cambio de personal, relevo o alerta.

3. Mapeo del Modelo de Datos (Firebase ■ SQL Server)

La migración transformará la base de datos documental (NoSQL) actual en un modelo relacional normalizado en SQL Server:

Colección Firebase (Actual)	Tabla SQL Server (Propuesta)	Tipo de Relación / Descripción
usuarios	dbo.Usuarios	Almacena credenciales (Hash) y roles (Coordinador, Supervisor).
personal_planta	dbo.Operarios	Ficha técnica de operarios con su turno asignado.
programa_produccion	dbo OrdenesProduccion	Carga de metas del Excel (SKU, cajas, botellas, fecha).
supervisores	dbo.Supervisores	Vincula al supervisor con su usuario e historial de líneas.
config_lineas	dbo.LineasAsignacion	Asignación de operarios a puestos específicos por fecha/turno.
ausencias	dbo.Ausencias	Historial de inasistencias y justificaciones por fecha.
metrics_plant	dbo.HistoricoOEE	Reporte consolidado de OEE, mermas y paros por turno.

4. Requerimientos de Red y Seguridad

Para el despliegue del nuevo backend, se requiere habilitar la siguiente configuración de infraestructura en la empresa:

Parámetro	Especificación Requerida
Servidor de BD	Microsoft SQL Server 2016 o superior. Base de datos dedicada con credenciales de lectura, escritura y ejecución.
Servidor de App	Entorno virtualizado (Windows Server IIS o Linux con Docker) para alojar la API REST en Node.js o .NET Core.
Puertos Internos	El puerto 1433 de SQL Server debe abrirse únicamente para la IP del Servidor de App. No expuesto a internet.
Acceso Clientes	Los puertos 80 (HTTP) y 443 (HTTPS) del Servidor de App deben ser accesibles desde la red Wi-Fi de la planta (VPN local).
Seguridad Web	Integración de certificado SSL corporativo en el Servidor de App para encriptación de datos en tránsito.

5. Plan de Transición y Cronograma

El proceso se dividirá en cuatro fases secuenciales para mitigar riesgos en producción:

- **Fase 1: Diseño y DDL (1 semana):** Diseño de esquemas de tablas relacionales en SQL Server y generación de scripts DDL. Migración inicial de datos de prueba.
- **Fase 2: Construcción de la API (2 semanas):** Desarrollo del backend en Node.js o .NET Core, endpoints de autenticación, mapeo de datos y configuración de WebSockets.
- **Fase 3: Integración y Refactorización (1.5 semanas):** Modificación del frontend en React para desviar las peticiones hacia la nueva API en lugar de Firebase SDK.
- **Fase 4: Despliegue y Pruebas QA (1 semana):** Instalación en el servidor de pruebas de la planta, pruebas de carga, verificación de sincronización offline y pase definitivo a producción.

Gobernanza y Control Completo: Al finalizar la migración, la empresa mantendrá la soberanía total de sus datos. No dependerá de servicios externos en la nube para el funcionamiento crítico de la planta, y el equipo de TI podrá monitorear el rendimiento del servidor y aplicar políticas de respaldos automáticos estándar directamente desde el Management Studio de SQL Server.